

Experiment No. – 1

Aim –

(A.) Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. **"Low Salary"**: All monthly salaries strictly less than \$20,000.

"Average Salary": All monthly salaries in the inclusive range [\$20,000, \$50,000].

"High Salary": All monthly salaries strictly greater than \$50,000. The final result table must report all three categories, even if a category contains zero accounts.

(B.) Design an Employee Management System database architecture by implementing the following operations sequentially:

1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns of employees, department and salaries, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below \$30,000.
3. List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

Tools Required

- Postgres
- PgAdmin4

Queries:

A.

```
CREATE TABLE Accounts (  
    account_id INT PRIMARY KEY,  
    income INT NOT NULL  
);  
  
INSERT INTO Accounts (account_id, income) VALUES  
(3, 108939),  
(2, 12747),  
(8, 87709),  
(6, 91738),  
(7, 45169);
```

```
SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count
FROM Accounts
WHERE income < 20000
UNION ALL
SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count
FROM Accounts
WHERE income >= 20000 AND income <= 50000
UNION ALL
SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count
FROM Accounts
WHERE income > 50000;
```

B.

```
CREATE TABLE Departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50) NOT NULL
);
CREATE TABLE Employees (
    emp_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    dept_id INT REFERENCES Departments(dept_id)
);
CREATE TABLE Salaries (
    emp_id INT PRIMARY KEY REFERENCES Employees(emp_id)
    ON DELETE CASCADE,
    salary INT NOT NULL
);

INSERT INTO Departments (dept_id, dept_name) VALUES
(10, 'HR'), (20, 'IT'), (30, 'Sales');
INSERT INTO Employees (emp_id, name, dept_id) VALUES
(1, 'Alice', 10), (2, 'Bob', 10), (3, 'Charlie', 20), (4, 'David', 20), (5, 'Emma', 30);
INSERT INTO Salaries (emp_id, salary) VALUES
(1, 50000), (2, 25000), (3, 80000), (4, 40000), (5, 45000);
```

```
SELECT e.name, d.dept_name, s.salary
FROM Employees AS e
JOIN Departments AS d
ON e.dept_id = d.dept_id
JOIN Salaries AS s
ON e.emp_id = s.emp_id;
```

```
UPDATE Salaries
SET salary = salary * 1.10
WHERE emp_id IN (
    SELECT e.emp_id
        FROM Employees e
    JOIN Departments d
    ON
        e.dept_id = d.dept_id
    WHERE d.dept_name = 'HR'
);
```

```
SELECT e.name, s.salary
FROM Employees AS e
JOIN Salaries AS s
ON e.emp_id = s.emp_id
WHERE s.salary > (SELECT AVG(salary) FROM Salaries);
```

Outputs:

```
CREATE TABLE Accounts (
  account_id INT PRIMARY KEY,
  income INT NOT NULL
);

INSERT INTO Accounts (account_id, income) VALUES
(3, 108939),
(2, 12747),
(8, 87709),
(6, 91738),
(7, 45169);

SELECT 'Low Salary' AS category, COUNT(account_id) AS accounts_count
FROM Accounts
WHERE income < 20000
UNION ALL
SELECT 'Average Salary' AS category, COUNT(account_id) AS accounts_count
FROM Accounts
WHERE income >= 20000 AND income <= 50000
UNION ALL
SELECT 'High Salary' AS category, COUNT(account_id) AS accounts_count
FROM Accounts
WHERE income > 50000;
```

Showing rows: 1 to 3

Page No: 1

	category text	accounts_count bigint
1	Low Salary	1
2	Average Salary	1
3	High Salary	3

Query

Query History

1 SELECT e.name, d.dept_name, s.salary
2 FROM Employees AS e
3 JOIN Departments AS d
4 ON e.dept_id = d.dept_id
5 JOIN Salaries AS s
6 ON e.emp_id = s.emp_id;
7
8
9
10
11
12
13
14

Data Output

Messages

Notifications

Showing rows: 1 to 5

Page No: 1 of 1

	name character varying (50)	dept_name character varying (50)	salary integer
1	Charlie	IT	80000
2	David	IT	40000
3	Emma	Sales	45000
4	Alice	HR	55000
5	Bob	HR	27500

Query

Query History

26
27
28
29
30
31
32 SELECT e.name, d.dept_name, s.salary
33 FROM Employees AS e
34 JOIN Departments AS d
35 ON e.dept_id = d.dept_id
36 JOIN Salaries AS s
37 ON e.emp_id = s.emp_id;
38
39
40

Data Output

Messages

Notifications

Showing rows: 1 to 5

Page No: 1 of 1

	name character varying (50)	dept_name character varying (50)	salary integer
1	Charlie	IT	80000
2	David	IT	40000
3	Emma	Sales	45000
4	Alice	HR	60500
5	Bob	HR	30250

Query

Query History

1 SELECT e.name, s.salary
2 FROM Employees AS e
3 JOIN Salaries AS s
4 ON e.emp_id = s.emp_id
5 WHERE s.salary > (SELECT AVG(salary) FROM Salaries);
6
7
8
9
10

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

	name character varying (50)	salary integer
1	Charlie	80000
2	Alice	60500

Learning Outcome

Understand table creation and relationships by designing databases with primary keys and foreign keys to maintain data integrity.

Develop proficiency in SQL queries using SELECT, JOIN, WHERE, UNION ALL, and aggregate functions like COUNT() and AVG() to retrieve meaningful information.

Learn to manipulate data by performing INSERT and UPDATE operations, including conditional updates using subqueries.